

Air University



Operating System (Lab)

Year 2026

Ms. Qurrat Ul Ain

Academic Year: 2024 – 2028

Department of Cyber Security

USAMA ARSHAD | 247141

BS – Cyber Security – Fall – 24 (A)

Date of Submission: April 24, 2026

Lab #08

Student Name Usama Arshad
Roll Number 247141
Team Number Team A
Task Number Lab Task 8

1. System Calls Used

System Call	Header	Purpose
shmget()	<sys/shm.h>	Create or access a shared memory segment
shmat()	<sys/shm.h>	Attach the segment to the process address space
shmdt()	<sys/shm.h>	Detach the segment from the process address space
shmctl()	<sys/shm.h>	Control / delete the shared memory segment
ftok()	<sys/ipc.h>	Generate a unique IPC key (alternative method)

2. Writer Process – Source Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/shm.h>

#define SHM_KEY 0x1234      /* Agreed-upon key shared by both processes */
#define SHM_SIZE 1024      /* Shared memory segment size in bytes */

int main(void)
{
    /* — Step 1: Generate / use the key */
    key_t key = SHM_KEY;
    printf("Writer Process\n");
    printf("=====\n");
    printf("Using IPC key: 0x%x\n", key);

    /* — Step 2: Create shared memory segment */
    int shmid = shmget(key, SHM_SIZE, IPC_CREAT | 0666);
    if (shmid == -1) {
        perror("shmget failed");
        exit(EXIT_FAILURE);
    }
    printf("Shared memory created. shmid = %d\n", shmid);

    /* — Step 3: Attach to shared memory */
    char *shm_ptr = (char *)shmat(shmid, NULL, 0);
    if (shm_ptr == (char *)-1) {
        perror("shmat failed");
        exit(EXIT_FAILURE);
    }
    printf("Attached to shared memory at address: %p\n", (void *)shm_ptr);
}
```

```
/* — Step 4: Take input from the user */
char message[SHM_SIZE];
printf("\nEnter message to write into shared memory:\n> ");
fgets(message, sizeof(message), stdin);

/* Remove trailing newline if present */
size_t len = strlen(message);
if (len > 0 && message[len - 1] == '\n')
    message[len - 1] = '\0';

/* — Step 5: Write message into shared memory */
strncpy(shm_ptr, message, SHM_SIZE - 1);
shm_ptr[SHM_SIZE - 1] = '\0'; /* guarantee null-termination */

/* — Step 6: Display confirmation */
printf("\n[Writer] Message written to shared memory: \"%s\"\n", shm_ptr);
printf("[Writer] Waiting for reader to read the message...\n");
printf("[Writer] Press ENTER to detach and exit.\n");
getchar(); /* pause so the reader can read before we exit */

/* — Step 7: Detach from shared memory */
if (shmdt(shm_ptr) == -1) {
    perror("shmdt failed");
    exit(EXIT_FAILURE);
}
printf("[Writer] Detached from shared memory. Exiting.\n");

return 0;
}
```

Explanation:

1. **Key:** Define a constant value, `SHM\_KEY = 0x1234`, which serves as the IPC key. Both the Writer and Reader need to use this identical key to access the same shared memory segment.
2. **shmget():** This function sets up a new shared memory area of 1024 bytes. It employs the `IPC\_CREAT` flag and permissions set to `0666` (allowing read/write access for all users). It returns an integer segment ID.
3. **shmat():** This function attaches the memory segment to the Writer's address space, providing a pointer to the beginning of the segment. You can pass `NULL` to allow the operating system to select the address automatically.
4. **User Input:** Utilize `fgets()` to safely gather a line of input from standard input (stdin) and store it in a local buffer.
5. **Write:** Use `strncpy()` to transfer the message into the shared memory area using the pointer obtained from shmat().
6. **Confirmation:** The program outputs the message to the user for verification before the Reader executes.
7. **shmdt():** This function detaches the memory segment from the Writer's address space, ensuring that the data remains intact until someone deletes the segment.

3. Reader Process – Source Code

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/shm.h>

#define SHM_KEY 0x1234      /* Must match the key used in writer.c */
#define SHM_SIZE 1024      /* Must match the size used in writer.c */

int main(void)
{
    /* — Step 1: Use the same key */
    key_t key = SHM_KEY;
    printf("Reader Process\n");
    printf("=====\n");
    printf("Using IPC key: 0x%x\n", key);

    /* — Step 2: Access the existing shared memory segment */
    int shmid = shmget(key, SHM_SIZE, 0666); /* 0 flags = access only, no create */
    if (shmid == -1) {
        perror("shmget failed (has the writer run yet?)");
        exit(EXIT_FAILURE);
    }
    printf("Shared memory accessed. shmid = %d\n", shmid);

    /* — Step 3: Attach to shared memory */
    char *shm_ptr = (char *)shmat(shmid, NULL, 0);
    if (shm_ptr == (char *)-1) {
        perror("shmat failed");
        exit(EXIT_FAILURE);
    }
    printf("Attached to shared memory at address: %p\n", (void *)shm_ptr);

    /* — Step 4: Read and display the message */
    printf("\n[Reader] Message read from shared memory:\n");
    printf("    \"%s\"\n", shm_ptr);

    /* — Step 5: Detach from shared memory */
    if (shmdt(shm_ptr) == -1) {
        perror("shmdt failed");
        exit(EXIT_FAILURE);
    }
    printf("\n[Reader] Detached from shared memory.\n");

    /* — Step 6: Mark shared memory for deletion
     * The reader cleans up the segment once it is done reading.
     * shmctl with IPC_RMID schedules removal; the segment is actually
     * deleted when the last process detaches. */
    if (shmctl(shmid, IPC_RMID, NULL) == -1) {
        perror("shmctl (IPC_RMID) failed");
        exit(EXIT_FAILURE);
    }
    printf("[Reader] Shared memory segment marked for deletion.\n");
    printf("[Reader] Exiting.\n");

    return 0;
}
```

Explanation:

1. **Same Key:** The Reader also makes use of the same key, `SHM\_KEY = 0x1234`, to locate the memory segment created by the Writer.

2. **shmget() (access only):** This call omits the `IPC\_CREAT` flag. If the Writer hasn't executed yet, this call will fail with `ENOENT`, displaying a helpful error message.
3. **shmat():** This function attaches the memory segment to the Reader's address space. Both processes will have their own pointers pointing to the same physical memory.
4. **Read & Print:** The Reader accesses the string directly from the shared pointer and prints it out.
5. **shmdt():** This function detaches the memory segment, making it inaccessible to the Reader.
6. **shmctl(IPC_RMID):** This schedules the segment for deletion, which will happen once all processes have detached from it, helping to conserve system resources.

4. Program Execution & Output

Compilation:

Writer:

```
usama@ubuntu:~/Desktop/Lab8$ vi writer.c
usama@ubuntu:~/Desktop/Lab8$ gcc writer.c -o writer
```

Reader:

```
usama@ubuntu:~/Desktop/Lab8$ vi reader.c
usama@ubuntu:~/Desktop/Lab8$ gcc reader.c -o reader
```

Permission Granting:

Writer

```
usama@ubuntu:~/Desktop/Lab8$ chmod +x writer
```

Reader

```
usama@ubuntu:~/Desktop/Lab8$ chmod +x reader
```

Writer Terminal:

```
usama@ubuntu:~/Desktop/Lab8$ ./writer
Writer Process
=====
Using IPC key: 0x1234
Shared memory created. shmid = 1
Attached to shared memory at address: 0x764e55cbb000

Enter message to write into shared memory:
> Hi there! This is Lab Task 8. I am in Team A.

[Writer] Message written to shared memory: "Hi there! This is Lab Task 8. I am in Team A."
[Writer] Waiting for reader to read the message...
[Writer] Press ENTER to detach and exit.

[Writer] Detached from shared memory. Exiting.
```

Reader Terminal:

```
usama@ubuntu:~/Desktop/Lab8$ ./reader
Reader Process
=====
Using IPC key: 0x1234
Shared memory accessed. shmid = 1
Attached to shared memory at address: 0x7fbff70b4000

[Reader] Message read from shared memory:
      "Hi there! This is Lab Task 8. I am in Team A."

[Reader] Detached from shared memory.
[Reader] Shared memory segment marked for deletion.
[Reader] Exiting.
```

5. Summary

In this lab, I explored Inter-Process Communication (IPC) using **System V Shared Memory** in C on Ubuntu Linux. I developed two programs—a Writer and a Reader—to facilitate data sharing without resorting to pipes or sockets. The Writer employs ``shmget()'` with the ``IPC_CREAT'` flag to establish a 1 KB shared memory area identified by the key ``0x1234'`. It then connects to this area using ``shmat()'`, accepts a message from the user, copies it into the shared space with ``strncpy()'`, and finally disconnects using ``shmdt()'`. The Reader, on the other hand, locates the same area using the same key, attaches to it, retrieves and displays the message, and subsequently detaches while scheduling deletion with ``shmctl(IPC_RMID)'` to reclaim system resources. This interaction takes place through a shared physical memory space managed by the OS kernel, making this method the fastest form of IPC, as it avoids data copying across different address spaces.

- End of Lab Task -